

# MÓDULO II

## Introdução à Programação II

# MÓDULO II

## Entrada e Saída de dados

Embora um computador possa dar solução a certos problemas muitas vezes mais rápido do que um ser humano, quais problemas do mundo real um **algoritmo** que não pode receber e nem apresentar dados pode resolver?

Por exemplo, é possível para um algoritmo informar nome e idade de um usuário, mas quais dados informar? Ele só saberá informar se receber esses dados. E o usuário só saberá que dados informar se o algoritmo pedir. Solucionamos esse impasse por meio da entrada e saída de dados.

A entrada/saída de dados compõem esse processo que consiste em receber os dados do mundo real, por meio do que é digitado ou da ação do mouse, por exemplo. Após isso, mostramos a informação processada ao usuário, o que pode acontecer por meio da tela, uma impressão ou ainda a gravação em um arquivo.

A seguir conheceremos o sistema de input/output de dados e como funciona a lógica, por isso, nesse primeiro momento não se preocupe com a escrita do código, pois dependendo da linguagem a grafia do comando altera.

# MÓDULO II

## Saída dos dados

Os algoritmos precisam interagir com o usuário e para isso podemos utilizar o comando **escreva()** para imprimir alguma coisa na tela. Esse é um comando muito simples e portanto será o primeiro que aprenderemos nessa mecânica de **entrada/saída de dados**.

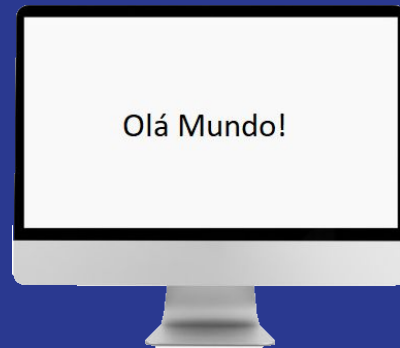
Por exemplo, podemos mostrar uma mensagem na tela utilizando o seguinte comando:

```
1 | escreva ("Olá mundo!")
```

Repare que após o comando, o dado a ser apresentado é colocado em parênteses.

Assim, conseguimos identificar o que é o comando e o que o compõe.

Contudo, lembre-se que agora a ideia é conseguir absorver a lógica de input/output de dados, por isso, não se atente ao uso de parênteses, mas sim que todas as informações vem depois do comando.



# MÓDULO II

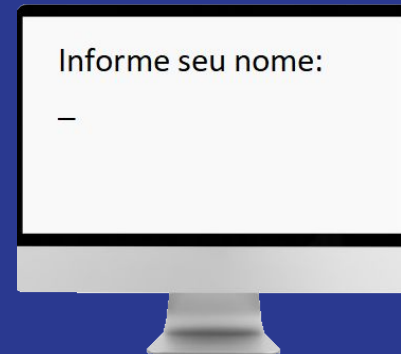
## Entrada de dados

Para que o usuário interaja com o algoritmo, este precisa ler o dado por meio do comando `leia`. Contudo, assim como quando lemos alguma informação e guardamos ela no nosso cérebro, o computador também guardará o dado lido na memória. Lembrando do conceito de variáveis, o dado informado será armazenado em uma variável.

```
1 | escreva ("Informe o seu nome:")  
2 | leia (nome)
```

Repare que na primeira linha temos o comando de saída `escreva`, que pede ao usuário que informe o seu nome. Sabemos disso por conta dos parênteses ;)

Já na segunda linha temos o comando de entrada `leia`, que vai pegar o dado informado e armazenar na variável `nome`. Assim como no comando `escreva`, tudo o que estiver entre parênteses será armazenado.



# MÓDULO II

Se quisermos saber o nome e o sobrenome do usuário separado por duas variáveis, devemos utilizar os seguintes comandos:

```
1 escreva ("Informe o seu nome e sobrenome:")  
2 leia (nome, sobrenome)
```

Nesse caso, é possível ler múltiplas variáveis de uma vez, separando-as variáveis por vírgula.

# MÓDULO II

## Retornando dados

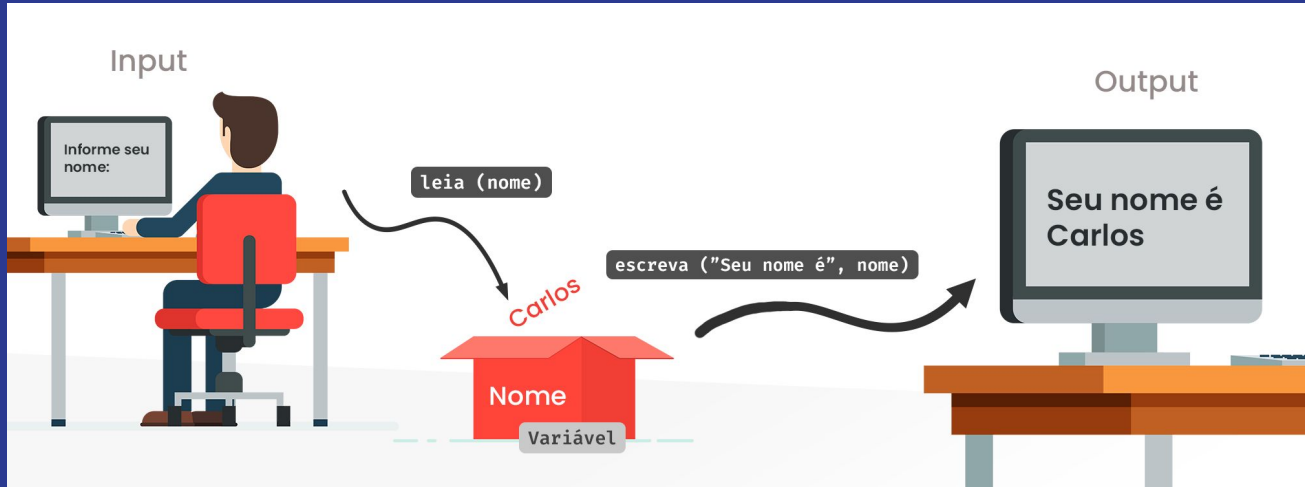
O comando escreva também pode retornar o valor de variáveis ao usuário, juntamente com outras informações que queremos passar.

Como exemplo podemos informar ao usuário qual foi o nome informado utilizando os seguintes comandos:

```
1 escreva ("Informe o seu nome:")  
2 leia (nome)  
3 escreva ("Seu nome é", nome)
```

Na **próxima figura** vemos como funciona o esquema, considerando que o usuário digitaria *Carlos* no momento da execução.

# MÓDULO II



Na linha 3 passamos a mensagem que queremos entre aspas e o algoritmo nos retorna o valor armazenado na variável nome. A sintaxe utilizada com a impressão de texto (**entre ""**) se faz necessária para dar clareza ao que está sendo impresso.

# MÓDULO II

## Leitura e Escrita de Dados em Formatos Comuns

### JavaScript Object Notation (JSON)

JSON (*JavaScript Object Notation*) é um formato de dados leve e de fácil leitura usado para armazenar e trocar informações estruturadas entre sistemas de computador. Ele é baseado em um subconjunto da linguagem de programação JavaScript e utiliza uma sintaxe simples de pares de chave-valor.

#### Escrita de JSON

Gerar um arquivo JSON para armazenar os dados gerados.

```
import json

dados = {
    "nome": "Alice",
    "idade": 25,
    "cidade": "São Paulo"
}

with open('dados.json', 'w') as arquivo:
    json.dump(dados, arquivo, ensure_ascii=False)
```



# MÓDULO II

## Comma-separated values (CSV)

CSV (Comma-Separated Values) é um formato de arquivo amplamente utilizado para armazenar dados tabulares, como planilhas ou tabelas, de maneira simples e legível. Nesse formato, os dados são organizados em colunas e linhas, onde cada linha representa um registro e as colunas representam diferentes atributos ou campos. Cada valor é separado por uma vírgula ou outro delimitador, como ponto e vírgula ou tabulação.

### Escrita de CSV

Gerar um arquivo CSV para armazenar os dados gerados

```
import csv

dados = [
    ['Nome', 'Idade'],
    ['Alice', 25],
    ['Bob', 30],
    ['Carol', 28]
]

with open('dados.csv', 'w', newline='') as arquivo:
    writer = csv.writer(arquivo)
    for linha in dados:
        writer.writerow(linha)
```

# MÓDULO II

## Microsoft Excel (XLSX)

XLSX é um formato de arquivo usado para armazenar planilhas eletrônicas, como as criadas no Microsoft Excel e em outros programas de planilha

### Escrita de XLSX

Gerar um arquivo XLSX para armazenar os dados gerados

```
import pandas as pd

dados = {
    'Nome': ['Alice', 'Bob', 'Carol'],
    'Idade': [25, 30, 28]
}

df = pd.DataFrame(dados)
df.to_excel('dados.xlsx', index=False)
```

# MÓDULO II

## **Evolução dos formatos de arquivos na Era dos Dados**

Como tudo na vida, o tempo trouxe mudanças. E no universo da tecnologia, essas mudanças vieram na forma de novos formatos de arquivos. Por quê? Para atender a uma variedade de necessidades, como armazenar mais dados, trabalhar mais rápido e otimizar análises.

Ao lidar com dados, esbarramos em muitos tipos de arquivos. Alguns são bem organizados, chamados dados estruturados e muito usados pelas empresas; os semi estruturados, que ainda é possível do ser humano ler; e os não estruturados, que estão se tornando mais populares devido a redes sociais e e-commerce.

# MÓDULO II

## Dados estruturados

**Ex.:** Banco de dados

**Estrutura rígida**  
Projetada previamente  
Representação homogêna

Cada campo de dados  
tem um formato bem  
definido.

Formato é um padrão  
aceito pelo campo.

Dados de um mesmo  
registro possuem  
relação entre eles.

Registros possuem  
valores diferentes, mas  
mesmos atributos.

Atributos ou campos  
são definidos por um  
esquema.

## Dados semi estruturados

**Ex.:** XML, JSON, RDF,  
OWL.

**Estrutura flexível**  
Representação heterogêna

Cada campo de dados  
tem uma estrutura,  
mas não existe uma  
imposição de formato

O esquema é criado  
com a definição de  
elementos internos dos  
arquivos (nós), legíveis  
para seres humanos

## Dados não estruturados

**Ex.:** Textos, arquivos,  
documentos, imagens,  
vídeos, áudios, redes  
sociais etc.

**Sem estrutura**  
(ou com estrutura mínima  
de arquivo)

Mais de 80% dos  
dados gerados no  
mundo é deste tipo

Você já deve ter ouvido falar de arquivos **TXT**, **CSV** e **JSON**, mas também existem outros que têm ganhado destaque: **AVRO**, **ORC** e **PARQUET**. Mas o que os torna especiais?

# MÓDULO II

## Tipos de arquivos

Durante a nossa vida, seja ela pessoal ou profissional, nos deparamos com uma enorme variedade de arquivos como textos, imagens e tabelas. Para cada tipo de arquivo existem diversos formatos que podemos utilizar. Por exemplo, para uma imagem, podemos salvar o arquivo como **jpeg, png, tiff**, etc...

O mesmo ocorre na área de dados, onde poderemos receber arquivos de nossos clientes nos mais variados formatos e caberá a nós, juntamente com todo o time de dados, escolhermos qual é o melhor formato para se trabalhar com cada tipo de dado com base em:

- Facilidade de uso
- Quantidade de dados que será armazenada
- Velocidade de escrita
- Velocidade de leitura
- Eficiência das consultas
- Grau de compressão

# MÓDULO II

Esses parâmetros são definidos com base na arquitetura de dados do projeto. Para isso, é necessário compreender as vantagens e desvantagens de cada formato de arquivo para determinar qual é a melhor extensão de arquivo a ser utilizada.

## **Formato do arquivos**

Dentre tantos formatos, além dos clássicos TXT e BIN os que mais se destacam são:

- CSV
- JSON
- AVRO
- ORC
- PARQUET

# MÓDULO II

Os arquivos eletrônicos normalmente são classificados como arquivos binários ou arquivos de texto. Pode-se argumentar que os arquivos de texto também são um tipo de arquivo binário porque, pois como qualquer outro arquivo eletrônico são formados por sequências de bytes. Porém, em geral, são considerados tipos diferentes.

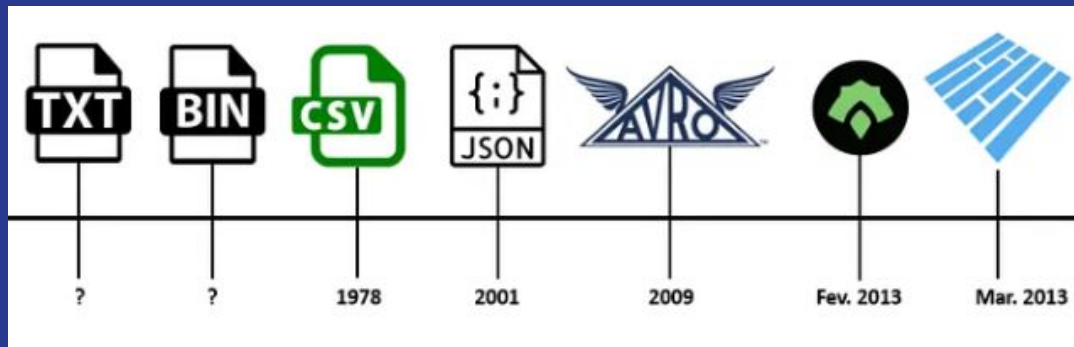
Podemos definir um arquivo binário como um arquivo que contém uma série de bytes sequenciais, onde cada um tem oito bits de comprimento. Esse formato não pode ser lido por um humano, é necessário um programa para interpretar os dados. Os arquivos binários incluem uma ampla variedade de tipos de arquivos, incluindo executáveis, bibliotecas, gráficos, bancos de dados, arquivos e muitos outros. Já os arquivos de texto estão em um formato legível e normalmente seguem conjuntos de caracteres específicos, como Unicode.

# MÓDULO II

Os arquivos **CSV** e **JSON** apesar de terem sido criados a mais de 20 anos ainda são os formatos mais vistos no mercado. Devido a familiaridade com ele, facilidade de interpretação e rapidez de escrita.

E no caso do **JSON** ainda existe o grande uso em **API** e mensagens.

Com o passar do tempo, as tecnologias foram evoluindo e entramos na era do Big Data, com aprimoramentos nas estruturas/formatos, os quais podem suportar estrutura de dados mais complexa, como a propriedade colunar e melhor compactação. E então, entre 2009 e 2013 foram criados os formatos **AVRO**, **ORC** e **PARQUET**, desenvolvidos pensando no processamento de grandes quantidades de dados.





# MÓDULO II

- **AVRO:** O Avro é um formato de arquivo de dados serializado, desenvolvido para o ecossistema do Apache Hadoop, que oferece eficiência na compactação e velocidade de acesso. Projetado para lidar com dados estruturados complexos, é particularmente útil para armazenar dados grandes distribuídos. Além disso, sua flexibilidade permite que esquemas de dados sejam alterados sem quebrar sistemas antigos.
- **ORC:** O ORC (Optimized Row Columnar) é um formato de arquivo auto-descritivo, altamente otimizado para leitura eficiente de grandes conjuntos de dados. Desenvolvido para o ecossistema Hadoop, ele combina a representação de dados em linha e em coluna (estrutura híbrida, para permitir análises rápidas e compactação superior. Além disso, suporta estruturas de dados complexas, como listas e mapas.
- **PARQUET:** O Parquet é um formato de arquivo colunar de código aberto, projetado para armazenamento eficiente e performático de dados estruturados em larga escala. Favorecido no ecossistema Hadoop e suportado por muitas outras plataformas de processamento de dados, ele oferece uma compactação de dados eficiente e a capacidade de realizar leituras rápidas, especialmente para consultas analíticas.

# MÓDULO II

Mas como devo escolher um desses arquivos? Quais as considerações devo fazer para definir qual formato utilizar? Bom nesse caso deve-se analisar as necessidades do seu projeto e sabendo quais são as características de cada formato definir a que mais se adequa ao projeto. Para isso verifique sobre:

## - Compressão de arquivos

A compressão é realizada por meio de diversos algoritmos de compressão, o quais irão reduzir a quantidade de bytes que representam um dado. A compressão dos dados também ajuda na remoção da redundância, onde por exemplo, “**BBBBBB**” que ocupa 6 bytes pode ser representada por ‘**6B**’, e passar a ocupar somente 2 bytes. Com isso há uma redução do tamanho do arquivo e consequentemente no armazenamento.

# MÓDULO II

## - Estrutura de armazenamento

Com relação ao tipo de armazenamento, a principal divisão é entre linha e coluna, sendo que mais recentemente tem-se falado do método híbrido, que é uma mistura de armazenamento em linha e coluna, mais conhecido como híbrido colunar ou até mesmo classificado somente como colunar.

Se dividirmos os formatos de arquivo citados, podemos dizer que:

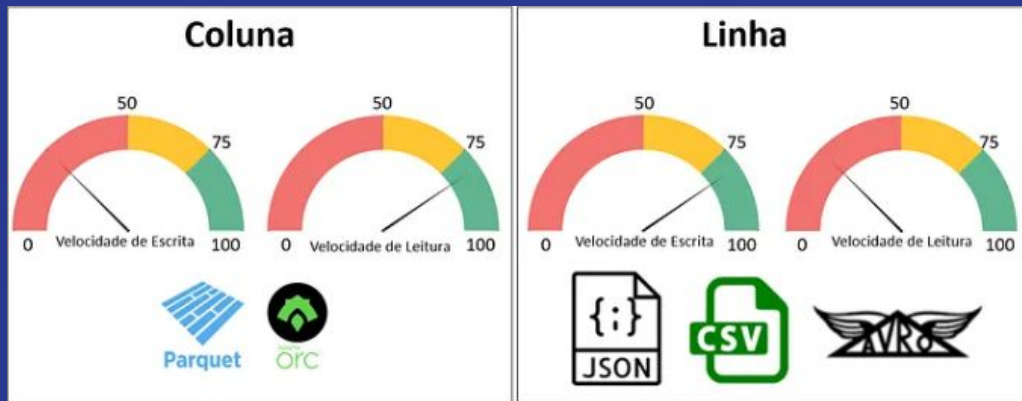
- Formato baseado em linha: **CSV**, **JSON** e **AVRO**
- Formato baseado em coluna: **ORC** e **PARQUET**.

# MÓDULO II

## Velocidade de escrita e leitura

Devido a diferença na estrutura de armazenamento, há mais vantagens na gravação ou leitura do arquivo e com isso deve-se pensar no objetivo do trabalho: “passarei mais tempo gravando ou lendo os arquivos”.

Os arquivos com formato baseado em linha são muito rápidos na gravação, porém muito lentos na leitura. Já os colunares são o oposto, sua gravação é mais demorada, porém a sua leitura é muito rápida.



# MÓDULO II

## Paralelismo

Devido a grande quantidade de dados que têm sido gerados atualmente é importante compreender se é possível dividir os dados entre múltiplos arquivos (“*Data Chunk*”). A facilidade de se realizar essa divisão será determinada pelo formato que o arquivo possui. Apesar de arquivos como **CSV** e **JSON** permitirem o paralelismo, a dificuldade de se executar é enorme.

Quando se trabalha em sistemas de arquivos distribuídos como o Hadoop, é primordial que os arquivos possam ser divididos em várias partes, pois assim o sistema poderá distribuir e operar seus arquivos utilizando técnicas de paralelismo. Caso o arquivo não possa ser dividido é necessário criar rotinas que façam esse processo, o que seria mais custoso ao time de engenharia.

# MÓDULO II

## Leitura Humana

Uma das características que pode determinar o formato a ser utilizado na gravação de um arquivo é se ele pode ou não ser lido por um humano. Dizem que por ser dados tabulares por exemplo já dá para um ser humano ler, porém deve-se considerar que existe uma diferença entre poder abrir um arquivo **CSV** diretamente e ter que criar uma *dataframe* ou a utilização de programas/sites preparados para a leitura de um arquivo parquet.

## Evolução de Shema

A evolução do schema é um recurso que permite aos usuários realizar automaticamente uma operação de acréscimo ou substituição de dados em um arquivo. Por exemplo, quando queremos adicionar uma nova coluna no nosso arquivo, isso irá fazer com que o schema do arquivo mude, para que ele possa comportar essa nova coluna.

# MÓDULO II

## Conclusão

Existem muitos tipos e formatos de dados que podemos utilizar na área de Dados, sendo que a escolha do formato ideal deve ser com base nas necessidades do projeto.

# MÓDULO II





# MÓDULO II

## Diferença entre Dados Estruturados e Não Estruturados

Existem 3 formas de classificar os dados de acordo com sua estrutura:

- Dados estruturados
- Dados semi-estruturados
- Dados não estruturados

A imagem acima mostra uma diferença visual, sugerindo que os dados **estruturados** são organizados em um padrão fixo, enquanto os **não estruturados** são seguem uma estrutura rígida. Os **semi-estruturados** fica entre os extremos: não são estruturados de forma rígida, mas também não são totalmente **desestruturados**.

# MÓDULO II

## Dados Estruturados

Dados estruturados são aqueles organizados e representados com uma **estrutura rígida**, a qual foi previamente planejada para armazená-los.

Pense em um formulário de cadastro com os campos: **nome**, **e-mail**, **idade** e uma **pergunta** que admite como resposta **sim** ou **não**. O campo **nome** será um **texto**, uma sequência de letras com ou sem a presença de espaços em branco, que terá um limite máximo e não poderá conter números ou símbolos. O campo **e-mail** também terá o padrão **textual**, mas formado por uma sequência de caracteres (e não só letras, pois admitirá números e alguns símbolos) e terá que ter obrigatoriamente um arroba. **Idade** é um campo que aceita apenas um **número** inteiro positivo, enquanto o campo referente a **pergunta** armazena um **valor binário** (pense um 1 bit, que pode ser 0 ou 1. Valor 0 para não, 1 para sim). Assim, cada campo possui um padrão bem definido, que representa uma estrutura rígida e um formato previamente projetado para ele. Destaco 2 pontos:

# MÓDULO II

Os dados de um mesmo cadastro estão relacionados (*dizem respeito a mesma pessoa*).

Em outras palavras, os dados **estruturados** de um mesmo bloco (*registro*) possuem uma **relação**.

Registros ou grupos de dados diferentes (*como de pessoas diferentes*), possuem diferentes valores, mas utilizam a mesma representação estrutural homogênea para armazenar os dados.

Ou seja, possuem mesmo **atributos** (*pense como sinônimo de campos no exemplo acima*) e **formatos**, mas valores diferentes.

Agora, veja, **banco de dados** é um exemplo de dados **estruturados**, mas existem outros. O **formulário** de cadastro, mesmo que salvasse os dados em outro recurso fora banco de dados (*como em um arquivo*), também é um exemplo de dados **estruturados** por conter campos definidos por uma estrutura rígida e previamente projetada, se enquadrando na definição.

# MÓDULO II

## Exemplos de Dados Estruturados

O exemplo mais típico de dados **estruturados** é um **banco de dados**. Nele, os dados são **estruturados** conforme a definição de um **esquema**, que define as tabelas com seus respectivos campos (*ou atributos*) e tipos (*formato*). O **esquema** pode ser pensado como uma meta-informação do **banco de dados**, ou seja, uma descrição sobre a organização dos dados que serão armazenados no banco. É exatamente como no exemplo do formulário que, normalmente, está interligado com um banco de dados.

# MÓDULO II

## Dados Não Estruturados

Qual é o oposto de uma estrutura **rígida** e previamente pensada? Uma estrutura **flexível** e **dinâmica** ou **sem estrutura**. Exemplo mais comum? Um documento ou um arquivo.

Pense em um arquivo feito em um editor de texto. Você pode adicionar quanto texto quiser, sem se preocupar com campos, restrições e limites. O arquivo pode conter também imagens, como gráficos e fotos, misturado com textos. Imagens, assim como vídeos ou arquivos de áudio, são também exemplos de dados **não estruturados**.

Assim, é fácil concluir que as redes sociais, as quais possuem um enorme volume de dados, como textos, imagens e vídeos criados diariamente por usuários, representam outro exemplo de dados **não estruturados**. Atualmente, mais de **80%** do conteúdo digital gerado no mundo é do tipo **não estruturado**.

# MÓDULO II

## Exemplos de Dados Não Estruturados

Normalmente, basta pensar em uma situação de dados que não seguem estrutura para termos exemplos de dados **não-estruturados**, mas é preciso tomar um pouco de cuidado com essa análise.

Em computação, todo dado, seja ele um arquivo ou um campo rígido, terá que ter algum tipo de estrutura, mesmo que mínima. Um **arquivo** é um tipo de estrutura mínima, pois é a unidade básica de armazenamento de um sistema operacional, mas ela é genérica, pois aceita diferentes tipos de dados.

Em resumo, quase tudo cairá em um arquivo, mesmo porque um vídeo tem que gravar em arquivo seus dados com um codificador (*codec*), um áudio também e assim por diante. Pensem, portanto, na estrutura interna do arquivo, se ela existe e é rígida, ou não.

# MÓDULO II

Assim, possivelmente, a maior parte dos arquivos que você pensar serão **não-estruturados**.  
Vamos aos exemplos:

**Textos diversos** - (*páginas da internet, relatórios, documentos, e-mails, mensagens em aplicativos como WhatsApp e etc...*)

**Imagens** - (*fotos, gráficos, ilustrações, desenhos e etc...*)

**Arquivos de áudio** - (*música, streaming e etc...*)

**Arquivos de vídeo** - (*filmes, seriados, feitos por usuários e etc...*)

**Redes sociais** - (*blogs, Facebook, Twitter, Instagram, LinkedIn e etc...*)

# MÓDULO II

## Diferença entre Dados e Não Estruturados

### Situação 1

**Situação 1:** *“Ao final de um evento presencial, cadastrarei pessoas interessadas em fazer um curso online com uma empresa. Para reservar uma vaga, preciso do nome, e-mail e idade da pessoa.”*

**Solução A:** Desenvolvi um sistema com campos em um formulário que cadastra os dados em um banco de dados. Planejei e criei previamente o banco de dados, o integrando com o sistema do formulário.

**Solução B:** Abri meu editor de texto e saí escrevendo os dados da galera. Coloquei cada dado em uma linha e tracei uma reta ao final de cada registro para separar as pessoas.



# MÓDULO II

## Situação 2

**Situação 2:** *“Durante o cadastro, algumas pessoas acreditavam que o curso seria presencial, no lugar que tinha ocorrido o evento e não online, de forma que, para esse público, eles preferem aulas presenciais. Entrei em contato com meu cliente, a empresa que solicitou o cadastro dos interessados, e eles então me pediram que eu armazenasse o endereço de todos os interessados para saber se eles moram perto de alguma outra unidade da empresa.”*

**Solução A:** Disse ao cliente que isso não seria possível. Não temos como inserir outra informação, como endereço, se não reformularmos o esquema do banco de dados para aceitar esse novo atributo, bem como editar a interface para inclusão de mais um campo, interligado ao atributo do banco. A estrutura de dados é rígida, por isso precisa ser previamente pensada e não será alterada na hora, em tempo real.

**Solução B:** Simples, criei uma linha a mais para cada registro e o editor de texto caiu como uma luva para essa demanda maluca em cima da hora. O arquivo de texto me permitiu qualquer tipo de cadastro, pois não impõe nenhuma estrutura interna rígida.

# MÓDULO II

## Situação 3

**Situação 3:** *“O cliente deseja obter os dados em forma de relatório, com o número total de cadastros e cada registro em uma linha da planilha.”*

**Solução A:** Simples, basta executar alguns comandos de banco de dados e teremos isso em segundos. O fato dos dados estarem estruturados ajuda muito tarefas como essa.

**Solução B:** Terei que contar manualmente, além de transpor manualmente para o formato de relatório também. Mesmo que eu faça um programa para fazer essa tarefa por mim, ainda assim, faremos uma busca em dados não estruturados.

# MÓDULO II

## **Importante**

É claro que forcei a barra no exemplo, pois a situação 1 e 2 correspondem a uma total falha de comunicação entre a divulgação do curso e os interessados. Na prática, no mundo real, a maioria dos cadastros são feitos com banco de dados e penso que seria bem improvável um fornecedor de tecnologia oferecer uma solução de cadastro em um editor de texto, como o Word ou o bloco de notas.

# MÓDULO II

## Dados Semi-Estruturados

Apresentam uma representação heterogênea, ou seja, possuem estrutura, mas ela é flexível. Assim, pensando no exemplo acima, ela agrega um pouco dos dados lados em termos de benefícios. Facilita o controle por ter um pouco de estrutura, mas também permite uma maior flexibilidade.

Um exemplo típico é um arquivo em **XML** (*eXtensible Markup Language*, que significa, em português, *linguagem de marcação estendida*), o qual possui nós, que são rótulos de abertura e fechamento, este precedido com o símbolo “/”, com os dados inseridos entre os nós. Vamos exemplificar.

Imagine o seguinte texto bruto (*digo, por estar em um editor de texto*):

*Nome: Leandro Pinho Monteiro*

*E-mail: leandro@mentalguild.com.br*

*Rua Siqueira Bueno, 1134, Mooca, São Paulo.*

# MÓDULO II

Agora, pensem nesses dados não-estruturados transpostos para um arquivo XML.  
O conteúdo do arquivo ficará assim:

```
<?xml version="1.0" encoding="UTF-8" ?>  
<cadastro>  
  <nome>Leandro Pinho Monteiro</nome>  
  <e-mail>leandro@mentalguild.com.br</e-mail>  
  <endereço>Rua Siqueira Bueno, 1134, Mooca, São Paulo</endereço>  
</cadastro>
```

A primeira linha serve para avisar que o arquivo contém uma estrutura XML e que usa codificação de caracteres unicode.

# MÓDULO II

Outros exemplos de arquivos com dados semi-estruturados:

- **JSON** – Javascript Object Notation,
- **RDF** – Resource Description Framework,
- **OWL** – Web Ontology Language.

Dados semi-estruturados possuem também outras classificações internas e características, mas esse assunto é mais avançado e será visto na temporada de banco de dados

# MÓDULO II



A Educação é o primeiro passo para um futuro melhor...

# MÓDULO II

Se você está interessado em desenvolver páginas web, certamente já ouviu falar sobre **HTML**, **CSS** e **JavaScript**. Essas três tecnologias desempenham papéis fundamentais no processo de criação de **sites**, permitindo a **estruturação** do conteúdo, **estilização** visual e adição de **interatividade**. Vamos mergulhar nas diferenças e características de cada uma dessas linguagens, e também explorar como elas trabalham em conjunto para produzir sites completos.

## O que é HTML?

**HTML**, abreviação para **HyperText Markup Language**, é uma linguagem de marcação utilizada para estruturar o conteúdo de uma página web. Ela define a hierarquia e a disposição dos elementos, como títulos, parágrafos, imagens e links. O **HTML** é responsável por organizar o esqueleto de um site, proporcionando uma base sólida para a estilização e interatividade posterior.



# MÓDULO II

## O que é CSS?

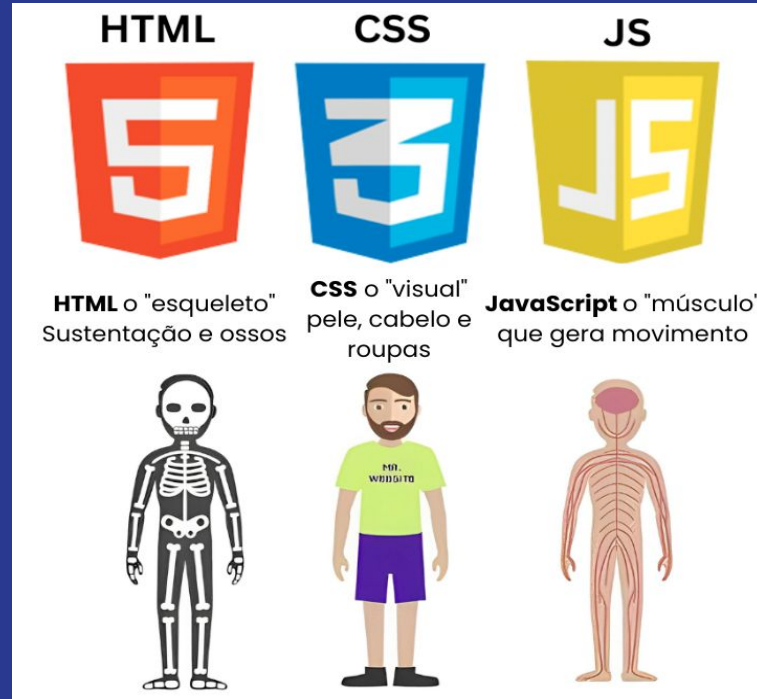
**CSS**, ou **Cascading Style Sheets**, é uma linguagem de folhas de estilo utilizada para definir a aparência visual e o layout de uma página web. Enquanto o **HTML** cuida da estrutura do conteúdo, o **CSS** se concentra na estilização desse conteúdo, permitindo a definição de cores, fontes, tamanhos e posicionamento dos elementos.

## O que é JavaScript?

**JavaScript**, por sua vez, é uma **linguagem de programação** que permite adicionar interatividade e dinamismo a uma página web. Com **JavaScript**, é possível criar funções, manipular elementos da página, realizar operações matemáticas e até mesmo fazer requisições a servidores para carregar dados externos. Essa linguagem desempenha um papel crucial na criação de experiências interativas para os usuários.

# MÓDULO II

## Diferença entre HTML, CSS e Javascript



# MÓDULO II

- **HTML** como o “esqueleto”: o **HTML** fornece a estrutura básica e a organização dos elementos em uma página web, como um esqueleto fornece a estrutura para o corpo humano.
- **CSS** como o “visual”: o **CSS** lida com a aparência e o estilo dos elementos **HTML**, assim como a pele, o cabelo e as roupas influenciam a aparência de uma pessoa.
- **JavaScript** como o músculo que gera movimento: Embora o **JavaScript** seja responsável por adicionar interatividade e dinamismo a uma página web, a comparação com os músculos é um pouco simplificada. Os músculos são responsáveis pelo movimento físico do corpo, enquanto o **JavaScript** é mais como o cérebro que controla a lógica e a interação na web. Além disso, o **JavaScript** não está diretamente relacionado à estética ou ao movimento visual da página.

# MÓDULO II



## HTML: A estrutura da web

### Como funciona o HTML

O **HTML** utiliza uma sintaxe simples de marcação, composta por elementos e tags (como o elemento de *quebrar/pular linha* **<br/>**), para estruturar o conteúdo de uma página web. Cada elemento é definido por uma tag, que pode ter atributos para fornecer informações adicionais. Essas tags podem ser aninhadas, formando uma hierarquia que indica as relações entre os elementos.

### Elementos básicos do HTML

No **HTML**, existem diversos elementos básicos que podem ser utilizados para estruturar uma página web.

Alguns exemplos são:

- **<html>**: a tag principal que define o início e o fim do documento **HTML**.
- **<head>**: a seção que contém informações sobre o documento, como o título da página e referências a folhas de estilo externas.
- **<body>**: a seção onde o conteúdo da página é inserido, como textos, imagens e links.

# MÓDULO II



## Tags HTML e sua funcionalidade

As tags **HTML** possuem diferentes funcionalidades, permitindo organizar o conteúdo de forma adequada.

Algumas tags comumente utilizadas são:

- **<h1>** a **<h6>**: utilizadas para definir os títulos da página, sendo **<h1>** o título mais importante e **<h6>** o menos.
- **<p>**: utilizada para definir parágrafos de texto.
- **<img>**: utilizada para inserir imagens na página, definindo o caminho para o arquivo de imagem e uma descrição alternativa.
- **<a>**: utilizada para criar links, definindo o destino do link e o texto que será exibido.

# MÓDULO II



## Semântica do HTML

Além de estruturar o conteúdo visualmente, o **HTML** também permite a utilização de elementos semânticos que fornecem um significado adicional para o conteúdo. Esses elementos ajudam os motores de busca e as tecnologias assistivas a compreenderem a estrutura e a finalidade do conteúdo.

Alguns exemplos de elementos semânticos são:

- **<header>**: utilizado para envolver o cabeçalho da página, geralmente contendo o logotipo e o menu de navegação.
- **<section>**: utilizado para agrupar conteúdos relacionados, como artigos ou blocos de texto.
- **<footer>**: utilizado para envolver o rodapé da página, geralmente contendo informações de copyright e links úteis.

# MÓDULO II



## CSS: Estilizando o conteúdo

### O papel do CSS na web

Enquanto o **HTML** cuida da estrutura e organização do conteúdo, o **CSS** se dedica a estilizar e dar vida visual a essa estrutura. Com o **CSS**, é possível definir **cores**, **fontes**, **tamanhos**, **espaçamentos** e até mesmo **animações**. Essa linguagem permite aos desenvolvedores controlar o aspecto estético de uma página web, criando uma experiência visual única e atraente para os usuários.

# MÓDULO II



## Propriedades básicas do CSS

O **CSS** possui uma vasta gama de propriedades que podem ser aplicadas a elementos **HTML**.  
Algumas das propriedades básicas mais comuns incluem:

- **color:** define a cor do texto.
- **font-size:** define o tamanho da fonte.
- **background-color:** define a cor de fundo de um elemento.
- **margin:** define as margens ao redor de um elemento.
- **padding:** define o espaço interno de um elemento.



# MÓDULO II



## Seletores CSS e sua aplicação

Os seletores **CSS** são usados para aplicar regras de estilo a elementos **HTML** específicos. Existem diferentes tipos de seletores, como seletores de elemento, seletores de classe e seletores de ID.

- Seletor de elemento: afeta todos os elementos do tipo selecionado.
- Seletor de classe: afeta todos os elementos que possuem a classe especificada.
- Seletor de ID: afeta apenas o elemento com o ID especificado.

## Box model e layout com CSS

O **CSS** utiliza o conceito de *box model* para definir a estrutura de um elemento, considerando o conteúdo, as margens, o preenchimento e as bordas. Essa estrutura permite controlar o *layout da página*, definindo as dimensões dos elementos e o espaçamento entre eles.

# MÓDULO II

## JavaScript: Interatividade na web



### O que é **JavaScript** e sua importância

**JavaScript** é uma linguagem de programação usada para adicionar interatividade e dinamismo a uma página web. Ela permite manipular elementos **HTML**, realizar operações matemáticas, validar formulários, fazer requisições a servidores e muito mais. O **JavaScript** é amplamente utilizado para criar experiências interativas e ricas para os usuários, tornando as páginas web mais dinâmicas e funcionais.

### Variáveis e tipos de dados em **JavaScript**

No **JavaScript**, as variáveis são utilizadas para armazenar valores. Existem diferentes tipos de dados que podem ser atribuídos a uma variável, como **strings** (*textos*), **números**, **booleanos**, **objetos** e **arrays**.

# MÓDULO II



## Funções e controle de fluxo em **JavaScript**

As funções desempenham um papel fundamental no **JavaScript**, permitindo agrupar um conjunto de instruções em um único bloco lógico. Além disso, o **JavaScript** possui estruturas de controle de fluxo, como loops e condicionais, que permitem executar diferentes partes do código com base em condições específicas.

## Diferenças entre **HTML**, **CSS** e **JavaScript**

Embora sejam tecnologias distintas, **HTML**, **CSS** e **JavaScript** trabalham em conjunto para criar páginas web completas e interativas. Enquanto o **HTML** estrutura o conteúdo, o **CSS** estiliza e dá vida visual a esse conteúdo, e o **JavaScript** adiciona interatividade e dinamismo.

# MÓDULO II



## Como essas tecnologias se complementam

**HTML**, **CSS** e **JavaScript** se complementam na medida em que cada uma dessas linguagens desempenha uma função específica. Ao combinar essas tecnologias corretamente, é possível criar sites ricos em recursos, visualmente atraentes e altamente interativos.

## Exemplos práticos de uso conjunto das três linguagens

Um exemplo prático de como as três linguagens trabalham juntas é o desenvolvimento de um formulário de contato. O **HTML** é utilizado para criar a estrutura do formulário, o **CSS** é responsável por estilizá-lo e torná-lo visualmente atraente, e o **JavaScript** pode ser aplicado para validar os campos do formulário e enviar os dados para um servidor.

# MÓDULO II

Desenvolvendo uma Página WEB de Teste de Forma Fácil e Rápida

## História do meu primeiro gatinho



O jubileu adora estar perto do dono, ama brincar e é muito carente.

A Educação é o primeiro passo para um futuro melhor...

# MÓDULO II

## Estrutura **HTML**

### **PASSO 1: Criando o arquivo em HTML**

Para desenvolver uma página web completa, o primeiro passo é criar a estrutura utilizando o **HTML**.

A estrutura deve incluir elementos de cabeçalho, corpo e rodapé, além de outros elementos necessários para o conteúdo específico da página.

Para isso, crie um novo arquivo em branco com a extensão .html: exemplo “gato.html”.

Você pode usar o Bloco de Notas ou qualquer editor de texto que preferir.

Dentro do arquivo **HTML**, comece com a estrutura básica do **HTML**:

# MÓDULO II

```
<!DOCTYPE html>
<html>
<head>
  <title>Minha Página sobre Gatos</title>
</head>
<body>

</body>
</html>
```

Explicação: Este código é como a “espinha dorsal” da sua página.

O **HTML** está entre as tags **<html>** **</html>**.

Já o **<head>** **</head>** contém informações sobre a página, como o título que aparece na aba do navegador.

Enquanto que o conteúdo da página está entre **<body>** **</body>**.

# MÓDULO II

## PASSO 2: Adicionando um título

Dentro do **<body>**, você pode adicionar um título para a sua página:

```
<h1>História do meu primeiro gatinho</h1>
```

Explicação: Usamos a tag `<h1>` `</h1>` para criar um título principal na página.



# MÓDULO II

## PASSO 3: Adicionando uma Imagem de Gato

Agora, você pode baixar a imagem e inserir o seguinte código para adicionar a imagem de um gato:

```

```

Explicação: Usamos a tag **<img>** para inserir uma imagem.

O atributo **src** especifica o caminho da imagem (*por exemplo*, “*gato.jpg*”) e o atributo **alt** fornece um texto alternativo que é exibido se a imagem não puder ser carregada.

PS: Insira a foto de um gato na mesma pasta que estiver o arquivo **.html**

# MÓDULO II

## PASSO 4: Descrevendo o Gato

Agora, adicione uma breve descrição sobre o gato:

```
<p>O jubileu adora estar perto do dono, ama brincar e é muito carente.</p>
```

Explicação: Usamos a tag `<p>` `</p>` para criar um parágrafo de texto.  
Aqui, explicamos algumas coisas sobre os gatos.

# MÓDULO II

## PASSO 5: Finalizando a Página

Finalmente, feche as tags **<body>** e **<html>**:

```
</body>  
</html>
```

Explicação: É importante fechar todas as tags que você abre para garantir que sua página funcione corretamente.

# MÓDULO II

## PASSO 6: É assim que deve ficar

Agora, seu arquivo “gato.html” deve se parecer com este código:

```
<!DOCTYPE html>
<html>
<head>
  <title>Minha Página sobre Gatos</title>
  <link rel="stylesheet" type="text/css" href="estilos.css">

</head>
<body>

  <h1>História do meu primeiro gatinho</h1>
  
  <p>O jubileu adora estar perto do dono, ama brincar e é muito carente.</p>
</body>
</html>
```

# MÓDULO II

## PASSO 7: Salvar e Visualizar

Salve o arquivo com a extensão **.html**, por exemplo, “***gato.html***“. Em seguida, abra-o em seu navegador para ver sua página em ação.

Explicação: Para ver sua página, basta clicar duas vezes no arquivo **HTML** que você criou. Seu navegador abrirá a página e você poderá ver o título, a imagem e a descrição que você adicionou.

# MÓDULO II

## Estilizando a página com CSS

Após criar a estrutura com HTML, é hora de estilizar a página com CSS. Isso envolve definir cores, fontes, espaçamentos e layouts adequados para alcançar o visual desejado. O CSS permite a criação de estilos únicos e personalizados para cada parte da página.

### **PASSO 1: Criando um arquivo CSS**

Crie um novo arquivo em branco com a extensão .css e salve-o na mesma pasta que o arquivo HTML.

Você pode dar o nome neste arquivo de “estilos.css”.

# MÓDULO II

## PASSO 2: Adicionando Estilos

Agora, vamos adicionar estilos CSS ao nosso HTML.

Dentro da tag <head> </head> no arquivo HTML, adicione o seguinte código para vincular o arquivo CSS:

```
<link rel="stylesheet" type="text/css" href="estilos.css">
```

## PASSO 3: Estilizando o Título

No arquivo “estilos.css”, adicione o seguinte código para estilizar o título:

```
h1 {  
  text-align: center;  
  color: #333;  
}
```

Explicação: Este código alinha o título ao centro e define a cor do texto como cinza escuro (#333).

# MÓDULO II

## PASSO 4: Estilizando a Imagem

Agora, estilize a imagem:

```
img {  
  display: block;  
  margin: 0 auto;  
  border: 2px solid #333;  
  max-width: 100%;  
  height: auto;  
}
```

Explicação: Isso faz com que a imagem seja exibida como um bloco centralizado com uma borda sólida cinza, com largura máxima de 100% para se ajustar ao tamanho da tela.



# MÓDULO II

## PASSO 5: Estilizando o Parágrafo

Estilize o parágrafo com o seguinte código:

```
p {  
  text-align: center;  
  font-size: 18px;  
  color: #666;  
  padding: 10px;  
}
```

Explicação: Isso centraliza o texto, define o tamanho da fonte como 18 pixels, a cor do texto como cinza médio (#666) e adiciona um pequeno preenchimento ao redor do parágrafo.

# MÓDULO II

## PASSO 6: Finalizando a Estilização

Agora, seu arquivo “*estilos.css*” deve se parecer com este código:

```
h1 {  
  text-align: center;  
  color: #333;  
}  
  
img {  
  display: block;  
  margin: 0 auto;  
  border: 2px solid #333;  
  max-width: 100%;  
  height: auto;  
}  
  
p {  
  text-align: center;  
  font-size: 18px;  
  color: #666;  
  padding: 10px;  
}
```

# MÓDULO II

## PASSO 7: Visualizar a Página

Abra seu arquivo HTML em um navegador e você verá sua página com os estilos aplicados.

Agora, seu conteúdo sobre gatos está bem apresentado e estilizado.

Lembre-se de que você pode personalizar ainda mais os estilos CSS conforme desejar!

## Adicionando interatividade com **JavaScript**

Para adicionar interatividade à página, o **JavaScript** entra em ação. É possível criar eventos, como cliques em botões, que desencadeiam a execução de funções **JavaScript** específicas. Essas funções podem manipular elementos **HTML**, fazer validações de formulários e realizar outras ações interativas. Não iremos entrar a fundo nesse quesito do desenvolvimento do código em **JavaScript** para não ficar muito extenso.

# MÓDULO II

## Onde utilizar as habilidades em HTML, CSS e JavaScript

As habilidades em **HTML**, **CSS** e **JavaScript**, além do conhecimento em **deploy**, são amplamente demandadas no mercado de trabalho atual.

Essas tecnologias são essenciais para a criação de sites e aplicações web, sendo utilizadas por empresas de todos os tamanhos e setores.

Além disso, desenvolvedores com conhecimento em **HTML**, **CSS**, **JavaScript** e na implantação eficaz de **deploy** também são procurados por agências digitais, startups e departamentos de TI.

Mas, devemos ressaltar que para você conseguir ter sucesso nessa área e conseguir um emprego, é preciso que você tenha conhecimento de mais tecnologias do **Front end** e **Back end** como **reactjs**, **nodejs**, **tailwind**, **nest**, versionamento de código dentre outras.

# MÓDULO II

## Carreiras relacionadas a essas tecnologias

Existem diversas carreiras relacionadas ao desenvolvimento web que exigem conhecimento em **HTML**, **CSS** e **JavaScript**. Algumas delas incluem:

- Desenvolvedor **front-end**: responsável pela implementação da interface visual de uma página web, utilizando **HTML**, **CSS** e **JavaScript**.
- Desenvolvedor **full-stack**: atua tanto no desenvolvimento do **front-end** como no do **back-end**, utilizando diversas linguagens e tecnologias.
- Web designer: responsável pela criação de layouts, imagens e estilos visuais para páginas web, utilizando ferramentas de design e conhecimentos em **HTML** e **CSS**.

# MÓDULO II

## Conclusão

### **Qual a diferença entre HTML, CSS e JavaScript?**

**HTML** é responsável pela estruturação do conteúdo, **CSS** pela estilização visual e **JavaScript** pela adição de interatividade e dinamismo.

### **É possível aprender HTML, CSS e JavaScript ao mesmo tempo?**

Sim, é possível aprender as três linguagens ao mesmo tempo, embora seja recomendado começar com **HTML** e **CSS** antes de mergulhar em JavaScript.

### **Quais recursos adicionais podem auxiliar no aprendizado dessas tecnologias?**

Existem diversos recursos disponíveis, como cursos online, tutoriais, livros e comunidades de desenvolvedores. Além disso, praticar a criação de projetos pessoais pode ser uma ótima maneira de consolidar o aprendizado.

# MÓDULO II

## É necessário conhecer todas as linguagens para desenvolver um site?

Sim, para desenvolver um site completo, é necessário ter conhecimentos básicos da linguagem de marcação de texto **HTML** e **CSS** além do **JavaScript**. No entanto, dependendo do tipo de projeto, podem ser necessárias outras tecnologias e linguagens adicionais.

## Quais são as possíveis dificuldades na aprendizagem do HTML, CSS e JavaScript?

Algumas dificuldades comuns podem incluir o entendimento da sintaxe, o gerenciamento do layout com **CSS** e o domínio dos conceitos mais avançados de **JavaScript**, como manipulação do DOM e requisições assíncronas. A prática e a persistência são fundamentais para superar essas dificuldades.

# MÓDULO II

## HTML: O Esqueleto da Casa

- Imagine o **HTML** como a planta baixa da casa. Ele define a estrutura, os cômodos, as portas e janelas.
- O **HTML** organiza o conteúdo da página, como *textos*, *imagens* e *vídeos*, usando "**tags**" (como se fossem etiquetas que identificam cada parte da casa).

### Exemplo:

- Uma tag **<p>** representa um parágrafo (um cômodo).
- Uma tag **<img>** representa uma imagem (uma janela com vista).
- Uma tag **<h1>** representa um título (o nome da casa).



# MÓDULO II

## CSS: A Decoração da Casa

- O **CSS** é como o designer de interiores da casa. Ele define a aparência, as cores, os estilos e o layout.
- O **CSS** controla como o conteúdo **HTML** é exibido, tornando a casa visualmente atraente.

### Exemplo:

- O **CSS** pode definir a cor das paredes (a cor do texto).
- O **CSS** pode definir o tamanho das janelas (o tamanho das imagens).
- O **CSS** pode definir o tipo de piso (a fonte do texto).

# MÓDULO II

## JavaScript: A Interatividade da Casa

- O **JavaScript** é como o sistema elétrico e hidráulico da casa. Ele adiciona interatividade e funcionalidades dinâmicas.
- O **JavaScript** permite que a casa responda às ações dos visitantes, como acender luzes, abrir portas e janelas, e até mesmo preparar um café.

### Exemplo:

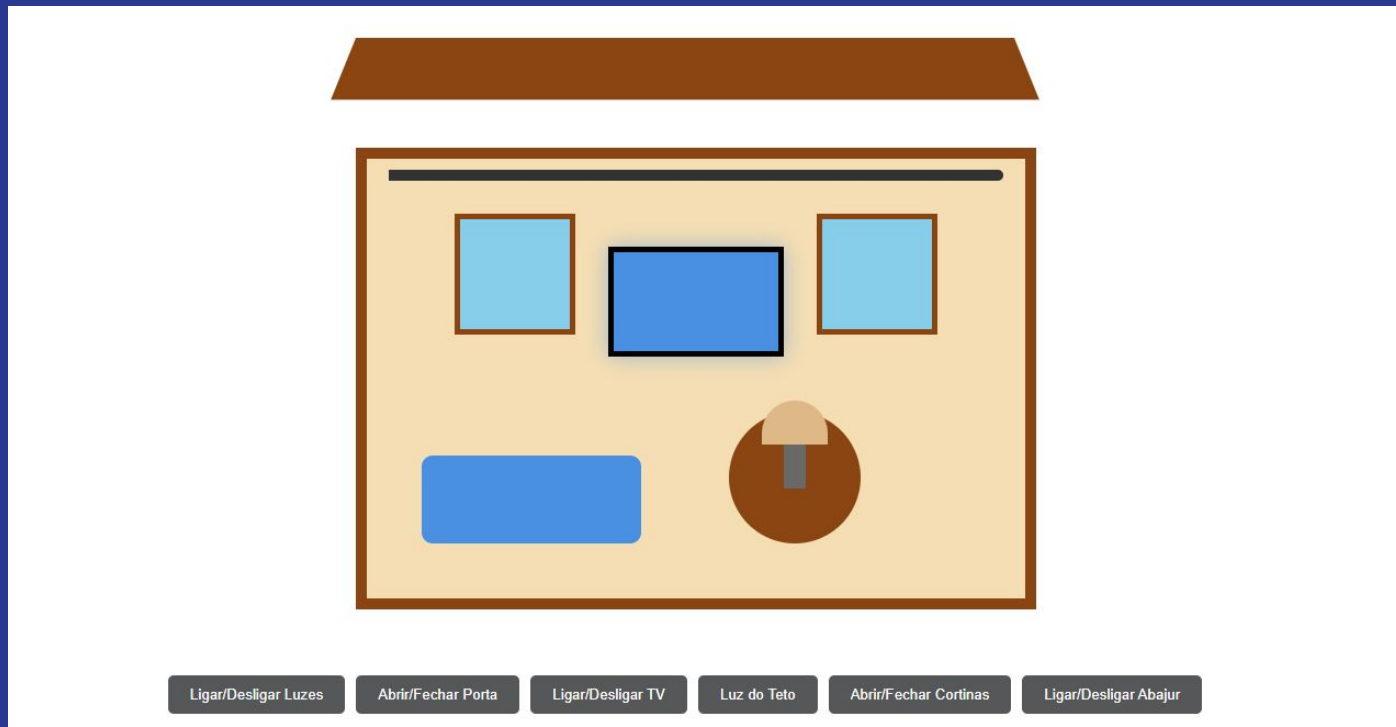
- O **JavaScript** pode fazer uma porta se abrir quando um botão é clicado.
- O **JavaScript** pode fazer as luzes acenderem quando anoitece.
- O **JavaScript** pode fazer uma janela exibir a previsão do tempo.

# MÓDULO II

## Em resumo:

- **HTML:** Define a estrutura e o conteúdo da página (o esqueleto da casa).
- **CSS:** Define a aparência e o estilo da página (a decoração da casa).
- **JavaScript:** Adiciona interatividade e funcionalidades dinâmicas (o sistema elétrico e hidráulico da casa).

# MÓDULO II



A Educação é o primeiro passo para um futuro melhor...

# MÓDULO II

## EXERCÍCIOS...

# MÓDULO II

FIM...